SpringBackend ESG

Projeto Cidades ESGInteligentes
Pipeline de CI/CD, containerização e orquestração

Integrantes
Nome Completo do Integrante 1 - RM000000
Nome Completo do Integrante 2 - RM000000
Nome Completo do Integrante 3 - RM000000

Repositório: https://github.com/CassioLinhares/ESG_javaSpring_devOps

1. Descrição do pipeline

Ferramenta: GitHub Actions, com o workflow `.github/workflows/ci-cd.yml`. Também são usados Maven Wrapper, Docker Buildx, GitHub Container Registry (GHCR), GitHub Environments e Docker Compose.

Gatilhos: push na `main`, pull request para a `main` e execução manual (`workflow_dispatch`).

```
Build → Testes → Imagem Docker → Deploy Staging → Deploy Produção
```

Etapas

Etapa	O que faz
Build	Configura o JDK 21 com cache do Maven, executa `./mvnw package` e publica o `.jar` como artefato.
Testes	Sobe um Oracle XE 21 como service container e executa `./mvnw test`. O teste carrega o contexto do Spring e aplica as 6 migrations do Flyway. Os relatórios do Surefire viram artefato.
Imagem Docker	Gera a imagem pelo Dockerfile multi-stage com cache do GitHub Actions e publica no GHCR com as tags do commit e `latest`.
Deploy Staging	Environment `staging`: baixa a imagem do commit, sobe app e banco com Docker Compose na porta 8081 e executa smoke test de cadastro e login.
Deploy Produção	Environment `production`: só roda após o staging passar. Repete o deploy na porta 8080 e o smoke test.

Lógica

- Cada job depende do anterior (`needs`). Uma falha no build ou nos testes impede a geração da imagem e os deploys.
- Em pull requests rodam build, testes e imagem, sem publicar nem fazer deploy. Em push na `main` a imagem é publicada e os deploys rodam em sequência.
- Credenciais vêm de secrets e variáveis dos GitHub Environments (`ORACLE_PASSWORD`, `DB_PASSWORD`, `JWT_SECRET`, `DB_USER`), com valores padrão quando não configurados. O environment `production` pode exigir aprovação manual.

2. Docker

Arquitetura

Recurso	Descrição
`app`	API Spring Boot (imagem `ghcr.io/cassiolinhares/esg_javaspring_devops`), inicia só depois do banco ficar saudável. Porta configurável por `APP_PORT`.
`oracle`	Oracle XE 21 (`gvenzl/oracle-xe:21-slim-faststart`) com o usuário da aplicação no PDB `XEPDB1` e healthcheck.
`oracle-data`	Volume nomeado com os dados do banco.
`esg-network`	Rede bridge dedicada; a app acessa o banco pelo nome `oracle`.
Variáveis	`.env` (modelo em `.env.example`): `APP_IMAGE`, `APP_PORT`, `ORACLE_PASSWORD`, `DB_USER`, `DB_PASSWORD`, `JWT_SECRET`. A app recebe `SPRING_DATASOURCE_*` e `MINHA_PALAVRA_SECRETA`.

Staging e produção usam o mesmo `docker-compose.yml` com projetos Compose diferentes (`esg-staging` e `esg-production`), então cada ambiente tem seus próprios containers, volume e rede.

Dockerfile

```
FROM eclipse-temurin:21-jdk AS build
WORKDIR /workspace
COPY .mvn .mvn
COPY mvnw pom.xml ./
RUN chmod +x mvnw && ./mvnw -B -q dependency:go-offline
COPY src src
RUN ./mvnw -B -q package -DskipTests

FROM eclipse-temurin:21-jre
LABEL authors="cassio linhares"
WORKDIR /app
RUN groupadd --system spring && useradd --system --gid spring spring
COPY --from=build /workspace/target/spring_backend_esg-0.0.1-SNAPSHOT.jar app.jar
USER spring
VOLUME /tmp
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Multi-stage: compila com JDK 21 e Maven Wrapper e leva apenas o `.jar` para uma imagem JRE 21, executada com usuário sem privilégios. O `pom.xml` é copiado antes do código para aproveitar o cache das dependências.

Comandos usados

```
docker build -t spring-backend-esg .
cp .env.example .env
docker compose up -d --build
docker compose ps
docker compose logs -f app
APP_PORT=8081 docker compose -p esg-staging up -d --no-build --wait
APP_PORT=8080 docker compose -p esg-production up -d --no-build --wait
docker compose down -v
```

Imagem criada

`ghcr.io/cassiolinhares/esg_javaspring_devops:84b9006d97abd11de5fdb38b206e9938c85e2280` (também publicada como `latest`), baseada em `eclipse-temurin:21-jre`.

3. Prints do pipeline rodando

GitHub Actions - CI/CD #4 (main)

```
Workflow: CI/CD  |  Execução #4  |  Evento: push  |  Branch: main  |  Commit: 84b9006
Status: completed  |  Resultado: success
https://github.com/CassioLinhares/ESG_javaSpring_devOps/actions/runs/38333744179

✓ Build             success       16s
✓ Testes            success     1m29s
✓ Imagem Docker     success       22s
✓ Deploy Staging    success     1m25s
✓ Deploy Produção   success     1m29s
```

Execução do workflow CI/CD na main com todas as etapas.

GitHub Actions - Build

```
Job: Build  |  Resultado: success  |  Runner: ubuntu-latest
https://github.com/CassioLinhares/ESG_javaSpring_devOps/actions/runs/38333744179/job/115089072888

✓ Set up job                                success      2s
✓ Run actions/checkout@v4                   success      0s
✓ Run actions/setup-java@v4                 success      1s
✓ Compilar e empacotar                      success      7s
✓ Run actions/upload-artifact@v4            success      3s
✓ Post Run actions/setup-java@v4            success      0s
✓ Post Run actions/checkout@v4              success      0s
✓ Complete job                              success      0s
```

Build: compilação e empacotamento.

GitHub Actions - Testes

```
Job: Testes  |  Resultado: success  |  Runner: ubuntu-latest
https://github.com/CassioLinhares/ESG_javaSpring_devOps/actions/runs/38333744179/job/115089144307

✓ Set up job                                success      1s
✓ Initialize containers                     success   1m02s
✓ Run actions/checkout@v4                   success      1s
✓ Run actions/setup-java@v4                 success      3s
✓ Executar testes                           success     17s
✓ Run actions/upload-artifact@v4            success      1s
✓ Post Run actions/setup-java@v4            success      0s
✓ Post Run actions/checkout@v4              success      0s
✓ Stop containers                           success      1s
✓ Complete job                              success      0s
```

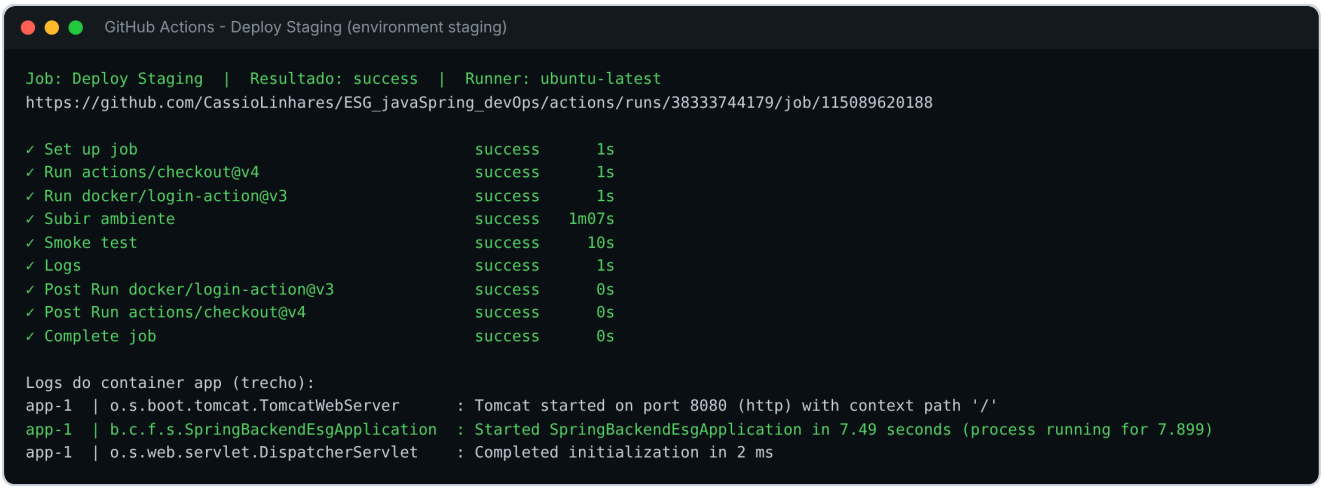
Testes: migrations do Flyway e teste executado contra o Oracle.

GitHub Actions - Imagem Docker

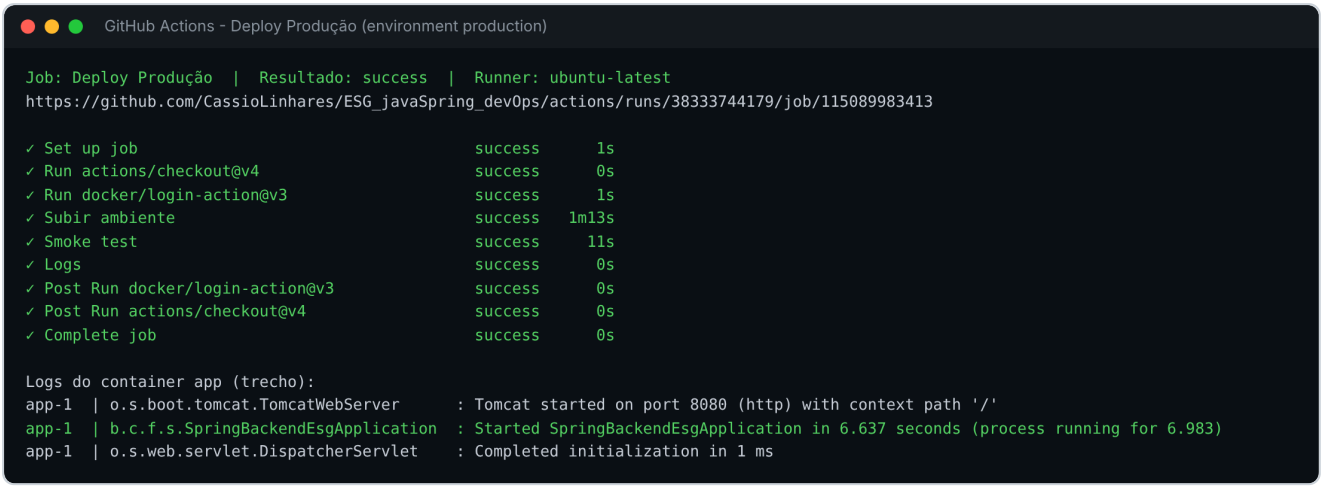
```
Job: Imagem Docker  |  Resultado: success  |  Runner: ubuntu-latest
https://github.com/CassioLinhares/ESG_javaSpring_devOps/actions/runs/38333744179/job/115089522140

✓ Set up job                                success      2s
✓ Run actions/checkout@v4                   success      1s
✓ Definir nome da imagem                    success      0s
✓ Run docker/setup-buildx-action@v3         success      5s
✓ Run docker/login-action@v3                success      1s
✓ Run docker/build-push-action@v6           success      6s
✓ Post Run docker/build-push-action@v6      success      2s
✓ Post Run docker/login-action@v3           success      0s
✓ Post Run docker/setup-buildx-action@v3    success      1s
✓ Post Run actions/checkout@v4              success      0s
✓ Complete job                              success      0s
```

Imagem Docker gerada e publicada no GHCR.

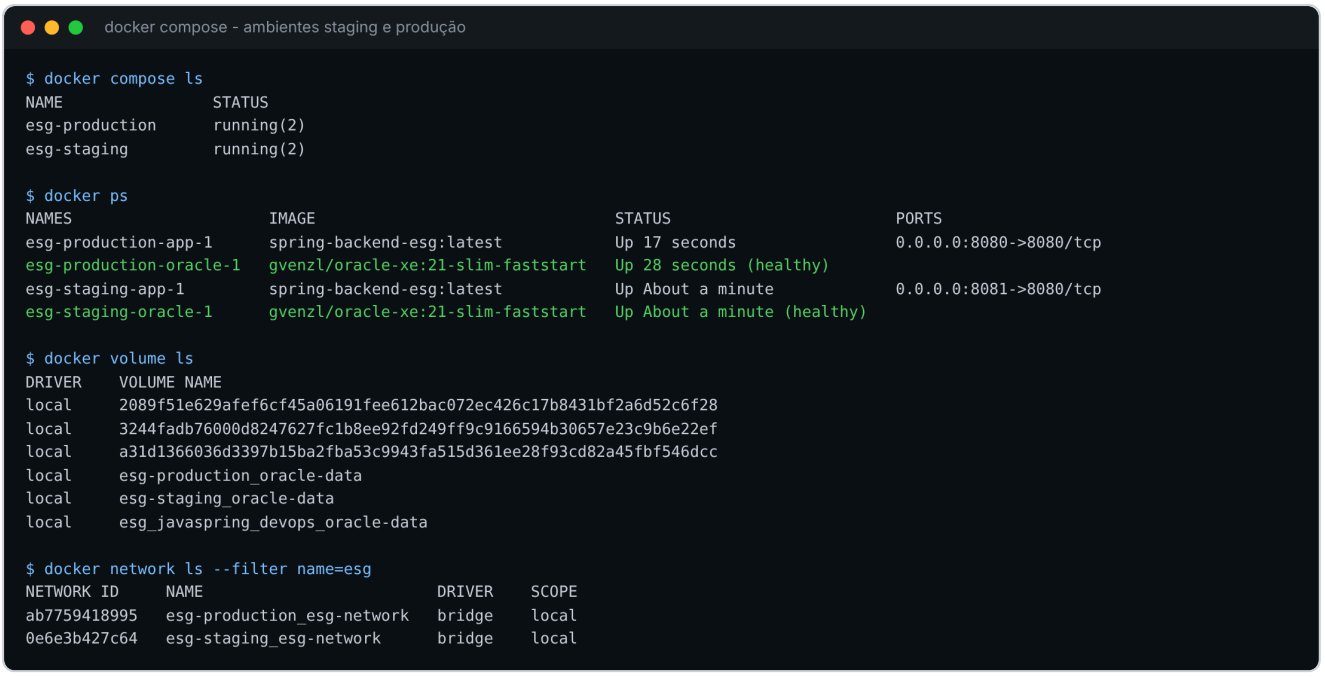


Deploy em staging com smoke test.



Deploy em produção com smoke test.

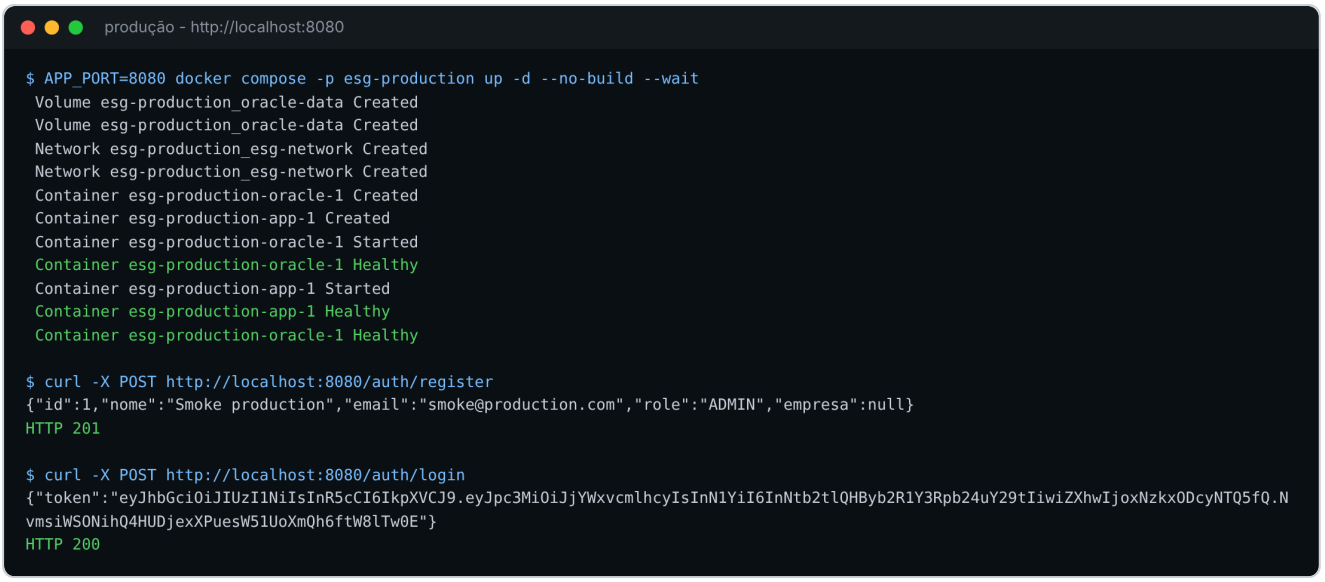
4. Ambientes staging e produção funcionando



Os dois ambientes rodando com Docker Compose, com volumes e redes separados.



Staging (porta 8081): cadastro e login respondendo.



Produção (porta 8080): cadastro e login respondendo.

5. Desafios encontrados e soluções

Desafio	Solução
O teste existente precisava de um Oracle em `localhost:1521`, indisponível no CI.	Oracle XE como service container no job de testes, com a conexão passada por variáveis de ambiente, sem alterar o código.
Rodar as migrations no schema `SYSTEM`, que não é vazio, faria o Flyway pular a V1 por causa do `baselineOnMigrate`.	Criação de um usuário próprio da aplicação (`APP_USER`) no PDB `XEPDB1`, com schema vazio, para que todas as migrations rodem.
Credenciais fixas no `application.properties`.	Uso do relaxed binding do Spring: `SPRING_DATASOURCE_*` e `MINHA_PALAVRA_SECRETA` sobrescrevem os valores, vindos do `.env` ou dos secrets dos environments.
O Dockerfile original dependia de um `.jar` gerado antes na máquina.	Dockerfile multi-stage que compila dentro da própria imagem.
O `mvnw` estava sem permissão de execução, o que quebra o build em Linux.	Permissão ajustada no Git e `chmod +x` no Dockerfile.
O Oracle demora para iniciar e a app falhava ao subir antes do banco.	Healthcheck no Oracle, `depends_on` com `service_healthy` e `docker compose --wait` nos deploys.
O GHCR exige nome de imagem em minúsculas e o repositório tem maiúsculas.	Conversão do nome com `tr` no job da imagem.
Não havia servidores de staging e produção disponíveis.	Deploy com Docker Compose no runner do GitHub, separado por GitHub Environments, com smoke test validando cada ambiente.

6. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	**OK**
Dockerfile funcional	**OK**
docker-compose.yml ou arquivos Kubernetes	**OK**
Pipeline com etapas de build, teste e deploy	**OK**
README.md com instruções e prints	**OK**
Documentação técnica com evidências (PDF ou PPT)	**OK**
Deploy realizado nos ambientes staging e produção	**OK**